Take-Home Assignment

KV Store

Overview:
Build a small, key–value service in three parts. High availability and redundancy are out of scope until the roadmap.

- Part 1 — Single-node (in-memory)
- Part 2 — Multi-node scale-out (capacity and performance)
- Part 3 — Technical roadmap (design-only)

Timebox: 3–5 hours of focused effort.
Make reasonable assumptions and note key tradeoffs.

Presentation format

You'll walk through your solution in approximately 40-50 minutes:
1. Demo each part (10–15 min): show the functionality working, explain your design choices and key tradeoffs
2. Code walkthrough (10–15 min): after seeing all parts in action, we'll dive into implementation details, discuss alternatives, and explore edge cases
3. Roadmap discussion (10–15 min): present your Part 3 technical plan
Come prepared to run your solution and talk through both what you built and why.

Tooling:

Use any development tools you like, including AI-assisted ones.
Part 1 — Single node

Build a single-process HTTP API that stores arbitrary JSON values by string key.
Storage: in-memory only for this stage.
Requirements:
Atomicity by key: concurrent operations on the same key are serialized; operations on different keys may proceed concurrently.
Concurrency safety: no torn reads/writes; no lost updates under parallel clients.
-Versioning: include a version per key.

CODA

Minimal API

- `GET /kv/{key}` → 200 `{ key, value, version }` or 404
- `PUT /kv/{key}` body: arbitrary JSON `value`; optional query/header for conditional update `ifVersion=<version>`
  - If `ifVersion` is supplied and does not match current version, return 409
  - On success, replace entire value and update version; return 200 `{ key, value, version }`
- `PATCH /kv/{key}` body: arbitrary JSON `delta`; optional `ifVersion=<version>` (same guard behavior as PUT)
  - If `ifVersion` is supplied and does not match current version, return 409
  - If key does not exist: create it with `delta` as value and initialize its version
  - If key exists and both existing value and `delta` are JSON objects: shallow-merge top-level fields, then update version
  - Otherwise: treat as replace (same as `PUT`), update version

Note: The `ifVersion` parameter acts as an optimistic lock guard for both PUT and PATCH operations. No version history is stored—only the current version matters.

Example usage (single node)

```bash
# Put a full object
curl -X PUT http://localhost:7000/kv/user:42 \
  -H 'Content-Type: application/json' \
  -d '{"name":"Ari","points":10}'
# Get it back
curl http://localhost:7000/kv/user:42
# Conditional replace (fail when version mismatches)
curl -X PUT 'http://localhost:7000/kv/user:42?ifVersion=1' \
  -H 'Content-Type: application/json' \
  -d '{"name":"Ari","points":20}'
# Upsert/merge (adds a field, version updates)
curl -X PATCH http://localhost:7000/kv/user:42 \
  -H 'Content-Type: application/json' \
  -d '{"rank":"gold"}'
```

Considerations

- How to test the implementation
- **Required test**: demonstrate 3 concurrent clients incrementing a counter at the same key (e.g., 100 increments each = final value 300)

Part 2 — Multi-node (scale out, memory capacity and performance)

Scale your service to use multiple processes so both the total memory available for keys/values and the overall throughput increase horizontally. Preserve part 1 semantics (per-key atomicity, versioning, API behavior). High availability and redundancy are not required.

Approach: your choice. For example, you may introduce a router, client-side partitioning, or any other design. Document your approach and how requests are routed.

Additional API requirement

- `GET /kv` → return a list of all keys across all nodes (NDJSON format: one line per key with `{"key": "some-key", "node": "node-id"}`)

Considerations:

- How keys/requests are distributed across nodes; how a client or router decides where to send a request.
- How the list keys endpoint aggregates keys from all nodes.
- How to test the implementation

Part 3 — Creative roadmap (design-only)

Pick an interesting roadmap item for this project and prepare a brief technical plan. Keep it high-level: a simple diagram and short notes on the change, expected benefits, important tradeoffs, and a rough effort/sequence. Choose something you find compelling for this system.

Do not implement changes for Part 3—design only.

By accepting and working on the assignments, the candidates agree and acknowledge that any assignments and materials provided to candidates are the property of Coda, including any of its subsidiaries and affiliates ("Coda"), and are intended for the sole use of the candidate in question during the recruitment process. The assignments have been modelled on Coda's older simulations and materials that are no longer in critical need. Coda does not have any current plans to use any part of the assignment submitted by the candidate, but retains the right to do so at its discretion in the future.